

Singular Value Decomposition (SVD)

Comprehensive Lecture Notes: Theory, Applications, and Implementation

Zamir Hussain
Department of Mathematics
University of Wah

February 10, 2026

🎓 Linear Algebra & Machine Learning Series 📖

Abstract

These comprehensive lecture notes cover Singular Value Decomposition (SVD), a fundamental matrix factorization technique with extensive applications in machine learning, dimensionality reduction, image processing, and data science. The notes bridge the gap between theoretical linear algebra and practical applications, addressing why traditional discrete concepts fail for real-world data and how SVD provides continuous measures of matrix properties. Included are step-by-step examples, mathematical derivations, practical applications, practice problems with solutions, and Python implementations.

Table of Contents

- | | |
|---|---|
| 1. 1. Introduction & Motivation | 7. 7. Low-Rank Approximation & Data Compression |
| 2. 2. The Problem with "Almost Singular" Matrices | 8. 8. Connection to PCA |
| 3. 3. The SVD Theorem & Definition | 9. 9. Applications in ML & Data Science |
| 4. 4. Computing the SVD: Step-by-Step | 10. 10. Practice Problems |
| 5. 5. Example 1: 2×2 Square Matrix | 11. 11. Solutions & Hints |
| 6. 6. Example 2: Rectangular Matrix | 12. 12. Summary & Key Takeaways |
| | 13. 13. Appendix: Python Implementation |

1 Introduction & Motivation

💡 Key Idea

The **Singular Value Decomposition (SVD)** bridges the gap between theoretical linear algebra and real-world data analysis by providing **continuous measures** where discrete concepts like rank and invertibility fail.

1.1 Why We Need SVD: Beyond Traditional Matrix Factorizations

While previous lectures covered matrix factorizations like LU and QR decompositions, SVD addresses fundamental limitations when dealing with real-world, noisy data:

Theoretical Concept	Real-World Reality	SVD Solution
Matrix is either invertible or singular	Matrices are <i>almost always</i> invertible but can be “ almost singular ”	Provides singular values quantifying “closeness” to singularity
Rank is discrete (full rank or rank- r)	Data matrices are technically full rank but “ almost low rank ”	Continuous measure of rank importance
Solutions are stable or unstable	Solutions can be highly sensitive to tiny errors	Identifies unstable directions via small singular values

Table 1: The gap between theory and practice in linear algebra

1.2 Three Powerful Applications Driving SVD Development

- Numerical Stability and Sensitivity Analysis:** SVD helps us quantify *how close* a matrix is to being singular, which is crucial for understanding how small errors in input data can lead to huge errors in the output.
- Data Compression and Low-Rank Approximation:** Many large data matrices (like user-movie ratings) are “**almost low rank**.” SVD provides the *optimal* way to factor such matrices into much smaller, manageable pieces.
- Dimensionality Reduction and PCA:** SVD is the mathematical engine behind **Principal Component Analysis (PCA)**, allowing us to find the best-fitting low-dimensional subspaces for complex, high-dimensional data points.

2 The Challenge with "Almost Singular" Matrices

2.1 1-Dimensional Example: Error Amplification

Example

Consider solving the simple equation $ax = b$:

- With $a = 0.002$ and $b = 1$, the exact solution: $x = 500$

- Realistic scenario: a measured with error ± 0.001
- Error bounds: $a \in [0.001, 0.003]$
- Solution bounds: $x = 1/a \in [333.33, 1000]$

Error Analysis:

$$\text{Relative error in } x = \frac{1000 - 333.33}{500} \times 100\% \approx 133\%$$

for only $\pm 50\%$ error in a .

Important Note

When a coefficient is close to zero, small measurement errors get catastrophically amplified. In higher dimensions, this problem is less obvious but more dangerous.

2.2 2-Dimensional Example: Nearly Parallel Columns

Example

Consider the system:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1.001 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

Analysis:

1. **Exact solution:** $x + y = 2$, $x + 1.001y = 2 \Rightarrow y = 0, x = 2$
2. **Perturbed system** (change second equation RHS by 0.001):

$$\begin{bmatrix} 1 & 1 \\ 1 & 1.001 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 2.001 \end{bmatrix} \Rightarrow 0.001y = 0.001 \Rightarrow y = 1, x = 1$$

3. **Dramatic shift:** 0.05% change in input $\rightarrow$ 100% change in solution

💡 Key Idea

The columns $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 1.001 \end{bmatrix}$ are almost parallel. The system is “**almost singular**”—the matrix has one very small singular value. SVD quantifies this instability through the **condition number**.

3 The Singular Value Decomposition Theorem

Theorem 1 (Singular Value Decomposition). *Any $m \times n$ matrix A with $\text{rank}(A) = r$ can be factored into:*

$$A = U\Sigma V^T$$

where:

- U is an $m \times m$ **orthogonal matrix** ($U^T U = I_m$). Its columns, $\mathbf{u}_i$, are the **left singular**

vectors. The first r columns, $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$, form an orthonormal basis for the column space of A .

- Σ is an $m \times n$ rectangular **diagonal matrix**. Its diagonal entries, σ_i , are the **singular values** of A . They are non-negative and ordered by convention: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, with $\sigma_{r+1} = \dots = 0$.
- V is an $n \times n$ **orthogonal matrix** ($V^T V = I_n$). Its columns, $\mathbf{v}_i$, are the **right singular vectors**. The first r columns, $\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$, form an orthonormal basis for the row space of A .

SVD Matrix Dimensions

Matrix	Symbol	Dimension
Original Matrix	A	$m \times n$
Left Singular Vectors	U	$m \times m$
Singular Values	Σ	$m \times n$
Right Singular Vectors (transposed)	V^T	$n \times n$

💡 Key Idea

[Geometric Interpretation] The SVD tells us that any linear transformation A can be broken down into three simple operations:

1. A **rotation** or reflection in the domain space (given by V^T)
2. A **scaling** along the new coordinate axes (given by Σ)
3. A **rotation** or reflection in the co-domain space (given by U)

The fundamental relationship connecting these components is:

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i \quad \text{for } i = 1, \dots, r$$

This means that A maps the i -th right singular vector to the i -th left singular vector, scaled by the i -th singular value.

4 Computing the SVD: Step-by-Step

4.1 Procedure for Finding U, Σ, V

1. **Compute V and Σ from $A^T A$:**
 - (a) Calculate the symmetric $n \times n$ matrix $B = A^T A$
 - (b) Find the eigenvalues λ_i and corresponding orthonormal eigenvectors of B
 - (c) The **right singular vectors** ($\mathbf{v}_i$) are these eigenvectors. Form $V = [\mathbf{v}_1 | \dots | \mathbf{v}_n]$
 - (d) The **singular values** (σ_i) are: $\sigma_i = \sqrt{\lambda_i}$. Form Σ with these values, ordered from largest to smallest
2. **Compute U :**

- (a) For all $\sigma_i > 0$, compute $\mathbf{u}_i = \frac{1}{\sigma_i} A \mathbf{v}_i$
- (b) If $m > r$, extend $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to a full orthonormal basis for $\mathbb{R}^m$

4.2 Detailed Calculation Example

Example

Find the SVD of $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$

✓ Step-by-Step Solution

Step 1: Compute $A^T A$

$$A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Step 2: Eigenvalues and eigenvectors of $A^T A$

- Characteristic equation: $(2 - \lambda)^2 - 1^2 = 0 \Rightarrow \lambda_1 = 3, \lambda_2 = 1$
- Singular values: $\sigma_1 = \sqrt{3}, \sigma_2 = 1$
- $\Sigma = \begin{pmatrix} \sqrt{3} & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$
- Eigenvectors:
 - For $\lambda_1 = 3$: $\mathbf{v}_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$
 - For $\lambda_2 = 1$: $\mathbf{v}_2 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$
- $V = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$

Step 3: Compute U

$$\mathbf{u}_1 = \frac{1}{\sqrt{3}} A \mathbf{v}_1 = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

$$\mathbf{u}_2 = \frac{1}{1} A \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

$$\mathbf{u}_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix} \quad (\text{from null space of } A^T)$$

$$U = \begin{pmatrix} 2/\sqrt{6} & 0 & 1/\sqrt{3} \\ 1/\sqrt{6} & -1/\sqrt{2} & -1/\sqrt{3} \\ 1/\sqrt{6} & 1/\sqrt{2} & -1/\sqrt{3} \end{pmatrix}$$

Final SVD: $A = U \Sigma V^T$

5 Example 1: 2×2 Square Matrix

Example

Given: $A = \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix}$

Task: Find SVD decomposition $A = U\Sigma V^T$

✓ Step-by-Step Solution

Step 1: Compute AA^T and $A^T A$

$$AA^T = \begin{bmatrix} 50 & 50 \\ 50 & 50 \end{bmatrix}$$
$$A^T A = \begin{bmatrix} 2 & 14 \\ 14 & 98 \end{bmatrix}$$

Step 2: Find Eigenvalues of $A^T A$

$$\det(A^T A - \lambda I) = (2 - \lambda)(98 - \lambda) - 196 = 0 \Rightarrow \lambda(\lambda - 100) = 0$$
$$\lambda_1 = 100, \quad \lambda_2 = 0$$

Step 3: Find Eigenvectors of $A^T A$

- For $\lambda_1 = 100$: $(A^T A - 100I)\mathbf{v}_1 = 0 \Rightarrow y = 7x \Rightarrow \mathbf{v}_1 = \begin{bmatrix} 1 \\ 7 \end{bmatrix}$
- For $\lambda_2 = 0$: $(A^T A - 0I)\mathbf{v}_2 = 0 \Rightarrow x = -7y \Rightarrow \mathbf{v}_2 = \begin{bmatrix} -7 \\ 1 \end{bmatrix}$

Step 4: Normalize Eigenvectors

$$\mathbf{v}_1^{\text{norm}} = \begin{bmatrix} 0.141 \\ 0.990 \end{bmatrix}$$
$$\mathbf{v}_2^{\text{norm}} = \begin{bmatrix} -0.990 \\ 0.141 \end{bmatrix}$$

Step 5: Construct V , Σ , and U

$$V = \begin{bmatrix} 0.141 & -0.990 \\ 0.990 & 0.141 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0.714 & -0.714 \\ 0.714 & 0.714 \end{bmatrix}$$

Step 6: Verification

$$U\Sigma V^T = \begin{bmatrix} 0.714 & -0.714 \\ 0.714 & 0.714 \end{bmatrix} \begin{bmatrix} 10 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0.141 & 0.990 \\ -0.990 & 0.141 \end{bmatrix} = \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix} = A$$

6 Example 2: Rectangular Matrix

Example

Given: $A = \begin{bmatrix} 1 & -2 & 3 \\ 3 & 2 & -1 \end{bmatrix}$

Task: Find SVD decomposition

✓ Step-by-Step Solution

Step 1: Compute $A^T A$

$$A^T A = \begin{bmatrix} 10 & 4 & 0 \\ 4 & 8 & -8 \\ 0 & -8 & 10 \end{bmatrix}$$

Step 2: Find Eigenvalues

$$\det(A^T A - \lambda I) = \lambda^3 - 28\lambda^2 + 180\lambda = 0$$

$$\lambda(\lambda - 10)(\lambda - 18) = 0 \Rightarrow \lambda_1 = 18, \lambda_2 = 10, \lambda_3 = 0$$

Step 3: Find Eigenvectors

- For $\lambda_1 = 18$: $\mathbf{v}_1 = \begin{bmatrix} -1/3 \\ -2/3 \\ 2/3 \end{bmatrix}$
- For $\lambda_2 = 10$: $\mathbf{v}_2 = \begin{bmatrix} 2/\sqrt{5} \\ 0 \\ 1/\sqrt{5} \end{bmatrix}$
- For $\lambda_3 = 0$: $\mathbf{v}_3 = \begin{bmatrix} -2/(3\sqrt{5}) \\ \sqrt{5}/3 \\ 4/(3\sqrt{5}) \end{bmatrix}$

Step 4: Construct V , Σ , and U

$$V = \begin{bmatrix} -1/3 & 2/\sqrt{5} & -2/(3\sqrt{5}) \\ -2/3 & 0 & \sqrt{5}/3 \\ 2/3 & 1/\sqrt{5} & 4/(3\sqrt{5}) \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \sqrt{18} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix}$$

$$U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}$$

Step 5: Verification

$$U\Sigma V^T = A$$

7 Low-Rank Approximation & Data Compression

Theorem 2 (Eckart-Young Theorem). *The best rank- k approximation to a matrix A (minimizing Frobenius or spectral norm error) is given by the truncated SVD:*

$$A_k = U_k \Sigma_k V_k^T$$

where U_k is $m \times k$, Σ_k is $k \times k$, and V_k^T is $k \times n$.

7.1 Practical Example: Netflix Recommendation System

💡 Key Idea

Consider a user-movie rating matrix with 10 million users and 10,000 movies:

	Full Matrix	Rank-10 Approximation	Improvement
Matrix entries	10^{11}	1.001×10^8	$\sim 1000\times$ fewer
Storage (8-byte)	~ 800 GB	~ 0.8 GB	1000$\times$ reduction
Operations ($y = Ax$)	$O(mn) = 10^{11}$	$O(mr + rn) = 2 \times 10^8$	500$\times$ faster
Computation time	~ 10 seconds	~ 0.02 seconds	500$\times$ speedup

Table 2: Efficiency gains from low-rank approximation

7.2 Compression Ratio Calculation

For an $m \times n$ matrix approximated by rank k :

- **Original storage:** mn entries
- **Compressed storage:** $mk + k + nk$ entries
- **Compression ratio:** $\frac{mk+k+nk}{mn}$

Example

Video Compression Example: A black-and-white video with:

- Resolution: 1920×1080 pixels
- 8-bit grayscale (1 byte per pixel)
- 30 minutes at 30 fps

✔ Step-by-Step Solution

1. Raw storage:

$$\text{Frames} = 30 \times 60 \times 30 = 54,000$$

$$\text{Pixels/frame} = 1920 \times 1080 \approx 2.07 \times 10^6$$

$$\text{Total bytes} \approx 54,000 \times 2.07 \times 10^6 \approx 1.12 \times 10^{11} \text{ bytes} \approx 112 \text{ GB}$$

2. Compressed (rank-100 approximation):

$$\text{Entries/frame} = (1920 \times 100) + 100 + (1080 \times 100) \approx 300,100$$

$$\text{Total numbers} \approx 54,000 \times 300,100 \approx 1.62 \times 10^{10}$$

$$\text{At 8 bytes/number: } \approx 1.3 \times 10^{11} \text{ bits} \approx 15.6 \text{ GB}$$

3. Compression ratio: $\approx 7 : 1$

8 Connection to Principal Component Analysis (PCA)

💡 Key Idea

The SVD is the mathematical engine behind Principal Component Analysis (PCA). When we have a set of high-dimensional data points, PCA aims to find a lower-dimensional subspace that best fits the data.

8.1 Comparison: Regression vs. PCA

Linear Regression	Principal Component Analysis
Goal: Model entire point cloud	Goal: Predict y from x
Distance minimized: Vertical distances	Distance minimized: Perpendicular distances
Model: $x \approx \alpha_1 u_1 + \alpha_2 u_2 + \dots + \alpha_r u_r$	Model: $y = f(x) + \epsilon$
Example: Fit line to (x, y) points	Example: Find plane that best contains 3D point cloud

Table 3: Fundamental differences between regression and PCA

8.2 SVD-PCA Relationship

Given centered data matrix X (each row is a data point):

- Right singular vectors ($\mathbf{v}_i$ in V) = **principal components**
- Singular values (σ_i) = **square roots of eigenvalues** of covariance matrix
- Left singular vectors (U) $\times \Sigma$ = **coordinates** in new basis

8.3 PCA Example

Example

A 2D dataset has covariance matrix: $C = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$
Find the principal components and variance explained.

✓ Step-by-Step Solution

1. **Eigenvalues:** $\det(C - \lambda I) = (10 - \lambda)^2 - 64 = 0 \Rightarrow \lambda_1 = 18, \lambda_2 = 2$

2. **Eigenvectors:**

- For $\lambda_1 = 18$: $v_1 = \frac{1}{\sqrt{2}}(1, 1)^T$
- For $\lambda_2 = 2$: $v_2 = \frac{1}{\sqrt{2}}(1, -1)^T$

3. **Variance explained:**

- First PC: $18/(18 + 2) = 90\%$
- Second PC: $2/(18 + 2) = 10\%$

4. **1D reconstruction** of point $(3, 4)$:

$$\text{proj} = ((3, 4) \cdot v_1)v_1 = \frac{7}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}(1, 1) = (3.5, 3.5)$$

9 Applications in Machine Learning & Data Science

Application	How SVD Helps	Key Benefit
Identifies small singular values $\rightarrow$ near-singular directions	Avoid catastrophic error amplification	Numerical Stability
Massive storage/computation savings	Truncate small singular values (assumed to be noise)	Low-rank approximation $A \approx A_r$
Noise Reduction		Cleaner signal, better models
Right singular vectors = principal components	Dimensionality reduction, feature extraction	PCA
Predict missing ratings efficiently	Approximate image matrix with few singular values	Low-rank completion of user-item matrix
Image Compression		JPEG-like compression with controllable quality
Solve $A^T A x = A^T b$ via SVD	Stable solution even for ill-conditioned systems	Linear Least Squares
Reveals semantic relationships between words	Natural Language Processing	Latent Semantic Analysis (LSA)

Table 4: Practical applications of Singular Value Decomposition

10 Practice Problems

Example

[title=Problem 1: Sensitivity Analysis] Consider the nearly singular system:

$$\begin{bmatrix} 2 & 2 \\ 2 & 2.001 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 4.002 \end{bmatrix}.$$

Tasks:

- (a) Solve the system exactly.
- (b) Change the second entry of RHS to 4.003 and solve again.
- (c) Compute the condition number $\kappa = \sigma_{\max}/\sigma_{\min}$.
- (d) Explain how the condition number relates to the sensitivity observed.

Example

[title=Problem 2: Low-Rank Approximation] Given the matrix

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix},$$

Tasks:

- (a) Show that A has rank 1.
- (b) Express A as uv^T where u, v are vectors.
- (c) Verify that the only nonzero singular value is $\sigma_1 = \sqrt{\text{sum of squares}}$.
- (d) What is A_{10} (rank-10 approximation)? Why?

Example

[title=Problem 3: Basic SVD] Given $A = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$, find its SVD decomposition.

Example

[title=Problem 4: Rectangular Matrix SVD] Find SVD of $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Example

[title=Problem 5: Image Compression Application] A 500×500 image has SVD with singular values: $[200, 150, 100, 50, 10, 5, 1, 0.5, \dots]$

- (a) How much storage is needed for rank-5 approximation?

- (b) What percentage of "energy" (sum of squared singular values) is preserved?

Example

[title=Problem 6: Computational Efficiency] Given $A = \begin{bmatrix} 2 & 2 \\ -1 & 1 \end{bmatrix}$:

- (a) Find its SVD
- (b) Use SVD to compute A^{10} efficiently

11 Solutions and Hints

11.1 Problem 1 Solution

✔ Step-by-Step Solution

- (a) **Exact solution:** Subtract equations $\rightarrow 0.001y = 0.002 \rightarrow y = 2, x = 0$
- (b) **Perturbed:** $0.001y = 0.003 \rightarrow y = 3, x = -1$
- (c) **Condition number:** Compute SVD to find σ_1 and σ_2 , then $\kappa = \sigma_1/\sigma_2$
- (d) **Sensitivity:** Large κ indicates high sensitivity to perturbations

11.2 Problem 2 Solution

✔ Step-by-Step Solution

- (a) **Rank:** All columns are multiples $\rightarrow \text{rank} = 1$
- (b) **Factorization:** $A = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$
- (c) **Singular value:** $\sigma_1 = \sqrt{1 + 4 + 1 + 4 + 16 + 4 + 1 + 4 + 1} = \sqrt{36} = 6$
- (d) **Rank-10:** Same as A since max rank is 3 (cannot have rank $> \min(m, n)$)

11.3 Problem 3 Hint

✔ Step-by-Step Solution

For diagonal matrix $A = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$:

- Singular values are absolute values: $\sigma_1 = 3, \sigma_2 = 2$
- U and V contain identity or sign adjustments

11.4 Problem 4 Solution Steps

✔ Step-by-Step Solution

1. Compute $A^T A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$
2. Eigenvalues: $\lambda_1 = 3, \lambda_2 = 1$
3. Singular values: $\sigma_1 = \sqrt{3}, \sigma_2 = 1$
4. Proceed with standard SVD procedure

11.5 Problem 5 Solution

✔ Step-by-Step Solution

(a) **Storage for rank-5:**

Original: $500 \times 500 = 250,000$ values
Compressed: $500 \times 5 + 5 + 500 \times 5 = 5,005$ values

(b) **Energy preserved:**

$$\frac{200^2 + 150^2 + 100^2 + 50^2 + 10^2}{\sum \sigma_i^2} \times 100\%$$

11.6 Problem 6 Hint

✔ Step-by-Step Solution

- (a) Compute SVD normally
- (b) Use property: $A^{10} = U \Sigma^{10} V^T$ (since U and V are orthogonal)
- (c) Σ^{10} is diagonal with entries σ_i^{10}

12 Summary & Key Takeaways

☰ Summary & Key Takeaways

Concept	Key Insight
SVD Formula	$A = U\Sigma V^T$: Rotation-Scaling-Rotation decomposition
Singular Values	$\sigma_i = \sqrt{\lambda_i(A^T A)}$, always non-negative, ordered
Orthogonality	$U^T U = I, V^T V = I$ (numerical stability)
Rank Revealing	# non-zero σ_i = rank of A
Condition Number	$\kappa(A) = \sigma_{\max}/\sigma_{\min}$ measures sensitivity
Low-Rank Approximation	$A_k = U_k \Sigma_k V_k^T$ (Eckart-Young Theorem)
Approximation Error	$\ A - A_k\ _F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$
PCA Connection	Right singular vectors = principal components
Image Compression	Store U_k, Σ_k, V_k^T instead of full matrix
Noise Reduction	Zero small σ_i corresponding to noise
Computational Note	Use $A^T A$ for V , AA^T for U (smaller dimension)

💡 Key Idea

[Fundamental Insights]

1. **SVD bridges theory and practice:** Provides continuous measures where discrete concepts fail
2. **“Almost” matters:** Real data lives in the continuum between full rank and low rank
3. **Efficiency gains:** Exploiting low-rank structure enables massive speed/storage improvements
4. **Numerical robustness:** SVD identifies and handles near-singular directions

12.1 Practical Guidelines

1. **Check singular values** before solving linear systems
2. **Truncate small singular values** for noise reduction and compression
3. Use **SVD-based methods** for ill-conditioned problems
4. **Monitor condition number** $\kappa = \sigma_{\max}/\sigma_{\min}$
5. **Always validate** with cross-validation for model selection

13 Appendix: Python Implementation

13.1 Basic SVD Computation

```
import numpy as np

def manual_svd(A):
    """Manual implementation of SVD for educational purposes"""
    # Compute  $A^T A$  and its eigenvectors
    ATA = np.dot(A.T, A)
    eigvals, eigvecs = np.linalg.eig(ATA)

    # Sort by eigenvalues (descending)
    idx = eigvals.argsort()[::-1]
    eigvals = eigvals[idx]
    V = eigvecs[:, idx]

    # Singular values
    sigma = np.sqrt(eigvals)

    # Construct Sigma matrix
    m, n = A.shape
    Sigma = np.zeros((m, n))
    min_dim = min(m, n)
    Sigma[:min_dim, :min_dim] = np.diag(sigma[:min_dim])

    # Compute U
    U = np.zeros((m, m))
    for i in range(min_dim):
        if sigma[i] > 1e-10:
            U[:, i] = np.dot(A, V[:, i]) / sigma[i]

    return U, Sigma, V.T

# Compare with NumPy's implementation
A = np.array([[1, 2], [2, 1]])
U_np, S_np, VT_np = np.linalg.svd(A, full_matrices=True)
print("Manual SVD:", manual_svd(A))
print("NumPy SVD U:", U_np)
print("NumPy SVD singular values:", S_np)
print("NumPy SVD  $V^T$ :", VT_np)
```

13.2 Low-Rank Approximation Functions

```
def low_rank_approximation(A, k):
    """Return rank-k approximation of A using SVD"""
    U, S, VT = np.linalg.svd(A, full_matrices=False)
    A_k = np.dot(U[:, :k], np.dot(np.diag(S[:k]), VT[:k, :]))
    return A_k

def compression_ratio(A, k):
    """Calculate compression ratio for rank-k approximation"""
    m, n = A.shape
```

```

original_storage = m * n
compressed_storage = m*k + k + n*k
return compressed_storage / original_storage

def reconstruct_from_svd(U, S, VT, k=None):
    """Reconstruct matrix from SVD components"""
    if k is None:
        k = len(S)
    return np.dot(U[:, :k], np.dot(np.diag(S[:k]), VT[:k, :]))

```

13.3 PCA Implementation using SVD

```

def pca_svd(X, n_components=2):
    """Principal Component Analysis using SVD"""
    # Center the data
    X_centered = X - np.mean(X, axis=0)

    # Perform SVD
    U, S, VT = np.linalg.svd(X_centered, full_matrices=False)

    # Principal components are columns of V (rows of VT)
    components = VT[:n_components].T

    # Project data onto principal components
    projected = np.dot(X_centered, components)

    # Explained variance ratio
    explained_variance = (S ** 2) / (X.shape[0] - 1)
    explained_variance_ratio = explained_variance / np.sum(explained_variance)

    return projected, components, explained_variance_ratio[:n_components]

# Example usage
X = np.random.randn(100, 10) # 100 samples, 10 features
X_projected, components, variance_ratio = pca_svd(X, n_components=2)
print("Explained variance ratio:", variance_ratio)

```

13.4 Image Compression Example

```

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def compress_image(image_path, k_values=[5, 20, 50, 100]):
    """Demonstrate image compression using SVD"""
    # Load image
    img = Image.open(image_path).convert('L') # Convert to grayscale
    A = np.array(img)

    # Perform SVD

```

```

U, S, VT = np.linalg.svd(A, full_matrices=False)

# Create subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.ravel()

# Original image
axes[0].imshow(A, cmap='gray')
axes[0].set_title(f'Original Image\nSize: {A.shape}')
axes[0].axis('off')

# Compressed images
for i, k in enumerate(k_values, 1):
    # Reconstruct using first k singular values
    A_k = np.dot(U[:, :k], np.dot(np.diag(S[:k]), VT[:k, :]))

    # Calculate compression ratio
    m, n = A.shape
    original_size = m * n
    compressed_size = m*k + k + n*k
    ratio = compressed_size / original_size

    # Plot
    axes[i].imshow(A_k, cmap='gray')
    axes[i].set_title(f'Rank-{k} Approximation\nCompression: {ratio:.2%}')
    axes[i].axis('off')

plt.tight_layout()
plt.show()

# Plot singular values
plt.figure(figsize=(10, 6))
plt.plot(S, 'b-', linewidth=2)
plt.xlabel('Index')
plt.ylabel('Singular Value')
plt.title('Singular Values of Image Matrix')
plt.grid(True, alpha=0.3)
plt.show()

return U, S, VT

# Usage example
# compress_image('sample_image.jpg')

```

13.5 Polynomial Fitting with Cross-Validation

```

import numpy as np
import matplotlib.pyplot as plt
from numpy.linalg import svd, cond, lstsq
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error

```

```

def polynomial_fitting_demo():
    """Demonstrate overfitting in polynomial regression"""
    np.random.seed(42)
    n = 50
    x = np.linspace(0, 2*np.pi, n)
    y_true = np.sin(x)
    noise = 0.1 * np.random.randn(n)
    y = y_true + noise

    degrees = [1, 3, 5, 10, 15, 20]
    coefficients = {}
    svd_results = {}

    # Perform polynomial fitting
    for degree in degrees:
        # Create Vandermonde matrix
        A = np.vander(x, degree + 1)

        # SVD analysis
        U, S, VT = svd(A, full_matrices=False)
        svd_results[degree] = {
            'singular_values': S,
            'condition_number': cond(A),
            'rank': np.sum(S > 1e-10)
        }

        # Solve using least squares (uses SVD internally)
        coeff, residuals, rank, s_values = lstsq(A, y, rcond=None)
        coefficients[degree] = coeff

        # Plot fitted polynomial
        x_fine = np.linspace(0, 2*np.pi, 200)
        A_fine = np.vander(x_fine, degree + 1)
        y_fitted = np.dot(A_fine, coeff)

        plt.plot(x_fine, y_fitted, label=f'Degree {degree}')

    plt.scatter(x, y, alpha=0.5, label='Data with noise')
    plt.plot(x, y_true, 'k--', linewidth=2, label='True function')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Polynomial Fitting with Different Degrees')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

    return coefficients, svd_results

# Run the demonstration
coeffs, svd_info = polynomial_fitting_demo()

```